

長庚大學
計算機網路

Computer Networks - Term project

C++ 網路爬蟲期末專題

課程教授: 李春良 教授

中華民國: 114年11月29日

學號: B1229062

姓名: 林冠妤

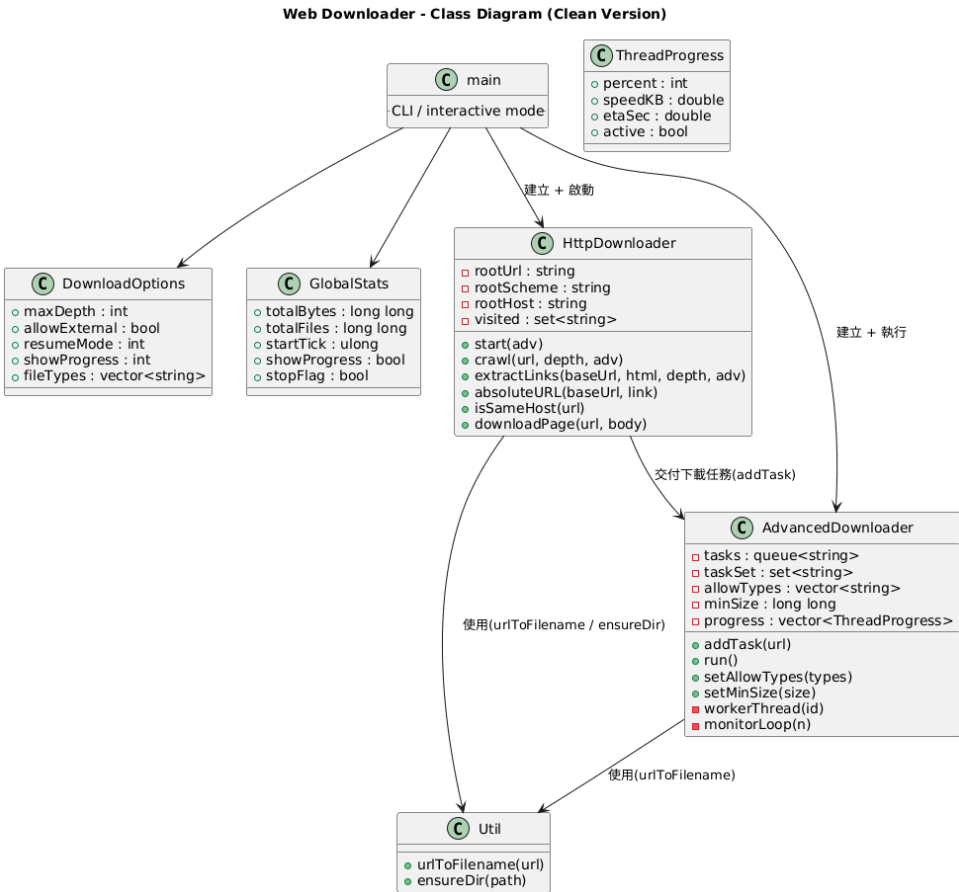
目錄

一、程式撰寫編排說明.....	2
二、完整程式碼說明(包含code & 註解).....	4
三、程式操作說明.....	35
四、成果(兩個測試).....	38
五、程式撰寫參考資訊.....	40
五、本次期末專題心得.....	43

一、程式撰寫編排說明

為了讓程式架構之完整的系統具有易維護性(好debug，方便加細節)，我將整體程式分成六個程式模組檔案，每一個都負責不一樣層級的邏輯與任務，且彼此之間也是互相關聯的，我將完整工作細項拆分成:main.cpp、util.h、downloader.cpp、downloader.h、advaced.cpp、advanced.h，六個程式模組架構。

以下是本程式期末專題，我的程式架構編排 UML 圖(圖(一)):



● 圖(一) 程式模組架構編排類別圖

再來是我的程式架構說明:

模組程式碼檔名	主要負責任務	負責細項
main.cpp	介面流程控制: 主程式控制 (程式入口/UI/參數處理)，管理流程、整合各模組	1. 讀取 user 輸入。 2. 處理命令列模式 & 互動模式。 3. 建立 DownloadOptions、GloabalStats、HttpDownloader (負責 HTTP 遞迴、擷取連結)、AdvancedDownloader (負責多執行緒抓檔案)。 4. 控制整體下載流程(start -> run -> 統計)。
util.h	共用工具: 負責提供所有檔案需要的共同工具函式	1. 工具函式1 - urlToFile:負責移除 http://、取最後一段 path、移除 query、過濾非法字元、只保留真正的檔名(不用 index.html)、確保外網不會互相覆蓋。 2. 工具函式2 - ensureDir:自動建立不存在的資料夾(新建立的資料夾)。

downloader.h	HTTP 遞迴下載、URL 解析： HTML 遞迴下載規格定義(class 宣告)	- 負責宣告 DownloaderOptions (下載選項結構)，讓主程式以及下載器互相交換設定，為設定中心。 - 以及宣告 GlobalStats (全域統計資料)，會由多執行緒共享，有用 mutex 保護。 - HttpDownloader (HTML 遞迴下載器)，專門負責HTML，不碰檔案下載。
downloader.cpp	HTTP 遞迴下載、URL 解析： HTML 遞迴下載器實作部分	- 包含完整的 HttpDownloader 的所有內容，是程式碼重要部分。 - 因為題目要求不行用現成的 HTTP 函式庫，我用了 WinSock、自己編排的 GET request，以及解析 Header / Body，以及字設定的 Content-Length。 - 解析 HTML 部分用 regex 抓 <a href> / 不依賴 parser。 - 我自己加的設計:將 URL 網址轉成絕對網址，像是: /img/a.jpg -> http://host/img/a.jpg - 重點-遞迴 crawl部分(深度限制、外網控制):照題目要求:做出深度為零時不遞迴，允許外站可抓取的限制 - 決定 HTML 是否要寫成檔案，避免多餘的檔案下載，我的判斷部分，使用者設定 (fileTypes) 判斷： ● 沒有限制: HTML 直接當檔案 ● 有限制但包含 html / htm :才寫 ● 否則只解析:不寫檔
advanced.h	多執行緒下載： 負責多執行緒下載器的宣告	- 負責： ● 任務 queue + mutex ● 支援同時 1~5 個 thread ● 每個 worker 的進度 (百分比、速度、ETA) ● monitor thread 監控所有進度 ● 設定檔案副檔名過濾 ● 設定檔案大小下限
advanced.cpp	多執行緒下載： 實作出最複雜的多執行緒下載器實作部分	- 任務管理(task queue):支援重複任務過濾(taskSet)，有 mutex 保護 queue，保障多執行緒安全去取任務 - workThread (真正下載檔案部分)，步驟： (1)解析 URL -> host/path (2)用 WinSock connect 到伺服器 (3)送出 GET 要求 (4)解析 header、找 Content-Length (5)顯示速度 / 百分比 / ETA (6)支援續傳 Range: bytes=xxx- (7)根據副檔名過濾 (8)根據 minSize 過濾 (9)寫入檔案，最後更新全域統計 (有用 mutex 保護) - monitorLoop (多執行緒及時做監控):會顯示每個 work 的進度(%)，以及下載速度(KB/s)。 - 設定暫停下載，按 q。

二、完整程式碼說明(包含code & 註解)

(一)程式有做的部分：

基本功能：

1. 提供兩種終端機模式供使用者輸入參數 (URL與輸出目錄)
 - (a) 命令列模式：使用者於執行時直接將必要的參數透過命令列輸入。
 - (b) 互動模式：程式執行後會輸出提示訊息要求使用者輸入參數。
2. 使用者可選擇的參數如下：
 - (a) 可選擇遞迴下載的深度 (深度為0即表示不遞迴下載)
 - (b) 可選擇是否下載外部網站 (即非原下載網站) 的內容
3. 顯示各物件的下載狀況 (物件資訊、進度、速度、預計剩餘完成時間)
4. 隨時更新目前的下載統計資訊 (已下載檔案總數量、已下載總容量、已下載時間)
5. 使用者可隨時停止下載，並且於之後選擇繼續下載 (已成功下載的物件不再下載) 或是重新下載 (從頭開始下載，已下載成功的物件先全部刪除)

進階功能：

1. 可同時下載多個檔案，並可設定同時下載檔案數上限
2. 可選擇下載的檔案類型 (如僅下載JPG檔案)
3. 可選擇下載的檔案大小限制 (如檔案需大於100 KB)

(二) 整體程式碼原創性說明：

本專案之六個程式檔案均由本人撰寫，並且依照project_l14.pdf的規格功能需求下去設計，並無抄襲他人程式碼，所有 Winsock HTTP 連線、遞迴下載、HTML 解析、絕對網址處理與進階多執行續邏輯皆為本人多次設計更改與撰寫。

專案中所參考使用的外部資源僅包含：

- 微軟官方 Winsock API 說明 (僅查詢函式用法，未抄程式碼)
- 微軟官方 filesystem API 文檔 (exists / create_directories)
- 官方 C++ regex 說明 (僅查語法，不抄程式碼)

其餘程式碼邏輯、設計、架構、命名為本人下去做更改設計。

(三) 特別撰寫的程式設計 & 程式資訊

- util.h:

1. 特別撰寫的程式設計

(1)urlToFilename()

- 自己寫的網址轉檔名演算法：
- 去掉 scheme
- 展開子目錄 (images/animals → images_animals)
- 過濾非法字元
- 過濾 html (不讓 html 被當非 HTML 檔案下載)
- 處理 query string

(2)ensureDir()

- 用官方 filesystem API 實作不存在資料夾去建立資料夾

(3)有自行設計避免下載 html 當一般檔案的規則

(4)自行定義 path 解決同名衝突

2. util.h 完整 code & 註解

```
// 避免這個檔案被include多次
#pragma once

#include <string>
// 用在檔案/資料夾操作
#include <filesystem>

// 1. urlToFilename() : 把 URL 轉成合法當檔名的字串
// 避免名字衝突被覆蓋掉以及避免非法字源造成無法存檔
inline std::string urlToFilename(const std::string &url) {
    std::string u = url;
    // (1) 去掉 URL 的 scheme 部分 (http:// 或 https://)
    size_t p = u.find("://");
    if (p != std::string::npos)
        u = u.substr(p + 3); // +3 是跳過 "://" 從後面開始取字串
    // (2) 找到第一個斜線 "/" 確認有 path
    // 像是 http://abc.com/docs/a.htm, 會找到 docs/a.htm 這個 path
    size_t slash = u.find('/');
    if (slash == std::string::npos)
        return ""; // 沒有 path 就不下載, 會無法當檔名會回傳空字串, 像是 http://abc.com 這種
    std::string path = u.substr(slash + 1);
    if (path.empty()) return ""; // 有 "/" 但後面沒有東西, 不當檔名
    // (3) 把所有 "/" 換成 "_", 改成安全的檔名
    // 像是 images/animals/cats/image.jpg, images_animals_cats_image.jpg
    for (char &c : path) {
        if (c == '/') c = '_';
    }
    // (4) 不阻擋 html 檔案下載, 但只抓 .htm, 不抓 .html (避免重複下載)
    // .html 負責解析網頁用, 不當下載檔案
    size_t dot = path.find_last_of('.');
    if (dot != std::string::npos) {
        std::string ext = path.substr(dot + 1);
        for (char &c : ext) c = tolower(c);
        if (ext == "html") return ""; // 不抓 .html 檔案
    }
    // (5) 把網址後面的 query 去掉, 避免檔名過長或有非法字元
    // 像是 image.jpg?version=1 會變成 image.jpg
    size_t q = path.find('?');
    if (q != std::string::npos)
        path = path.substr(0, q);
    // (6) 移除其他非法字元, 像是: * , ? , " , < , > , |
    for (char &c : path) {
        if (c == ':' || c == '*' || c == '?' || c == '"' ||
            c == '<' || c == '>' || c == '|' || c == '\\')
            c = '_';
    }
}
```

```

// (7) 回傳安全的檔名字串
return path;
}

// 2. 如果資料夾不存在會自動建立資料夾
inline void ensureDir(const std::string &path) {
    if (!std::filesystem::exists(path)) {
        // 可以建立多層資料夾
        std::filesystem::create_directories(path);
    }
}
}

```

● downloader.h:

1. 特別撰寫的程式設計

(1) DownloadOptions

- 客製化: 最大深度、外部網站、續傳、顯示進度、檔案篩選等

(2) GlobalStats

- 自己寫的統計資料共享結構(多執行緒安全)

(3) class HttpDownloader

- HTML 遞迴邏輯
- parseURL
- absoluteURL
- isSameHost
- crawl()、extractLinks()

(4) 自行設計 visited set 避免重複下載，以及URL 判斷 html / 非 html 的邏輯

2. downloader.h 完整 code & 註解

```

// 避免這個檔案被include多次
#pragma once

#include <string>
#include <vector>
// 用 set 避免網址重複加入
#include <set>
// 避免多執行緒衝突
#include <mutex>

// 1. user 設定的下載選項
struct DownloadOptions {
    int  maxDepth = 0; // 遞迴深度(0 = 只抓起始頁)
    bool allowExternal = false; // 是否允許去抓外部網站
    int  resumeMode = 0; // 0 = 正常, 1 = 續傳, 2 = 刪掉就有紀錄重新下載
    int  showProgress = 0; // 是否顯示進度
    // user 限制下載的檔案類型, 如果這邊 vector 是空的就要全部下載
    // 然後 HTML 也會計入 totalFiles 中

```

```

    std::vector<std::string> fileTypes;
};
// 2. 全域統計的資訊 (基礎 part + 進階 part 一起共用)
struct GlobalStats {
    long long totalBytes = 0; // 下載的 total bytes
    long long totalFiles = 0; // 下載的 total 檔案數量
    unsigned long long startTick = 0; // 開始下載的時間(算總耗時用)
    bool showProgress = true; // 是否顯示進度
    bool stopFlag = false; // 是否要停止下載

    std::mutex mtx; // 保護這些變數的 mutex
};
// 宣告 AdvancedDownloader 的類別，因為後面的 HttpDownloader 參數會用到
class AdvancedDownloader;
// 3. HttpDownloader 類別: 用來解析 html、爬超連結、抓 html 頁面
class HttpDownloader {
public: // 建構子: 設定起始的 URL、輸出的資料夾、下載的各個選項、全域統計之資料
    HttpDownloader(const std::string &startUrl,
                   const std::string &outDir,
                   const DownloadOptions &opt,
                   GlobalStats* stats);

    // 開始爬網頁(遞迴執行 crawl), adv = 進階下載器，用來下載非 html 檔案
    void start(AdvancedDownloader* adv);
    // 把 link (相對路徑) 轉成絕對 URL
    // 像是 base = http://abc.com/docs/index.htm;
    // link = ../image/a.jpg; 會回傳 http://abc.com/image/a.jpg
    std::string absoluteURL(const std::string &baseUrl,
                           const std::string &link);
    // 判斷這個 URL 網址是不是跟起始網址同一個 host (避免抓外部網站)
    bool isSameHost(const std::string &url);
    // 回傳起始網址的 host 部分，像是 hsccl.fr.to
    std::string getRootHost() const { return rootHost; }

private:
    // 解析 URL，拆成各個部分: scheme, host, port, path
    // 像是 http://abc.com:80/docs/a.htm
    // scheme = http; host = abc.com; port = 80; path = /docs/a.htm
    bool parseURL(const std::string &url,
                  std::string &scheme,
                  std::string &host,
                  int &port,
                  std::string &path);
    // 遞迴抓 HTML, depth = 目前深度
    void crawl(const std::string &url,
               int depth,
               AdvancedDownloader* adv);
    // 從 HTML 內容中找出所有連結，baseUrl = 這份 HTML 的 URL(用來處理相對路徑)

```

```

void extractLinks(const std::string &baseUrl,
                 const std::string &html,
                 int depth,
                 AdvancedDownloader* adv);
//下載 HTML 本體，url = 要下載的網址，body = 回傳 HTML 內容字串
bool downloadPage(const std::string &url, std::string &body);
private:
std::string rootUrl; // 起始 URL 網址(用來遞迴比較)
std::string outDir; // 輸出資料夾
std::string rootScheme; // http / https (作業只有支援 http，https 只是為了處理連結)
std::string rootHost; // 起始 host (用來判斷有無抓外部網站)
DownloadOptions opt; // user 所有下載選項(遞迴深度、續傳、檔案類型等等)
std::set<std::string> visited; // 已經下載過的 URL 網址(避免重複下載)
GlobalStats* stats; // 全域統計的資料(下載量、檔案數量等等)
};

```

● downloader.cpp:

1. 特別撰寫的程式設計

(1) 核心邏輯

- HTTP GET 請求字串組合
- 手動 connect / send / recv
- Header parsing + Content-Length 抓取
- 第二次連線下載 body
- HTML link 擷取 (自己的 regex + 自己的遞迴 crawler)
- html rewrite (將 src/href 換成本地檔)
- 避免外網 html 覆蓋，host_index.htm 有特別處理
- 決定 html 要不要寫檔(取決 fileType 與使用者設定)

(2) 用 regex 搜尋 link

(3) 避免 external site 覆蓋，自行作 host 前綴

(4) 寫兩階段連線：先取長度再取 body

(5) stopFlag 中斷機制(支援 q 中止)

2. downloader.cpp 完整 code & 註解

```

// HttpDownloader 類別定義
#include "downloader.h"
// AdvancedDonloader，多執行緒下載器
#include "advanced.h"
// urlToFilename 等工具函式
#include "util.h"

//Window socket 必備
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>

```

```

#include <conio.h>
// 連 socket 函式庫
#pragma comment(lib, "ws2_32.lib")

#include <fstream>
#include <iostream>
#include <filesystem>
// 使用 regex 找 href/src
#include <regex>
// setw/format 用
#include <iomanip>

using namespace std;
namespace fs = std::filesystem;

// getExtLower:小工具，從 URL 裡抓出路徑的副檔名(小寫、沒有點)
static std::string getExtLower(const std::string &url) {
    // 1. 去掉 http:// 或 https://
    size_t p = url.find("://");
    size_t start = (p == string::npos ? 0 : p + 3);
    // 2. 找出 path
    size_t slashPos = url.find('/', start);
    string path = (slashPos == string::npos ? "/" : url.substr(slashPos));
    // 3. 去掉 ? 後面的 query string
    size_t q = path.find('?');
    if (q != string::npos)
        path = path.substr(0, q);
    // 4. 最後一個點後視為副檔名
    size_t dot = path.find_last_of('.');
    if (dot == string::npos) return "";
    string ext = path.substr(dot + 1);
    // 5. 轉小寫
    for (char &c : ext) c = tolower(c);
    return ext;
}

// isHtmlLike:判斷 URL 是否應該當作 HTML 下載/解析
static bool isHtmlLike(const std::string &url) {
    std::string ext = getExtLower(url);
    // 只有 .htm 才算 HTML(避免重複下載)
    if (ext == "htm") return true;
    // path 有斜線但沒有副檔名，是資料夾，回傳 true (HTML)
    // 例如 http://hsccl.us.to/
    // 但是不能把沒有 slash 的網址當 HTML!
    size_t p = url.find("://");
    size_t start = (p == std::string::npos ? 0 : p + 3);
    size_t slash = url.find('/', start);
    // 沒有 path '/'，不能當 HTML (例如 http://hsccl.fr.to)

```

```

    if (slash == std::string::npos) return false;
    // 如果 path 存在但沒有副檔名， HTML 資料夾
    if (ext == "") return true;

    return false;
}
// rewriteHTML:將 HTML 內的所有 src/href 連結改成本地檔名(離線瀏覽使用)
// 像是 img src="images/cat1.jpg" 變成 img src="images_cat1.jpg"
static std::string rewriteHTML(const std::string& html,
                               const std::string& baseUrl,
                               HttpDownloader* dl)
{
    std::string out = html;
    // 抓出所有 src="xxx" 或 href="xxx"
    std::regex rg("(src|href)\\s*=\\s*\"([^\"]+)\"", std::regex::icase);
    std::smatch m;

    std::string result;
    std::string::const_iterator it = html.begin();
    std::string::const_iterator ed = html.end();

    while (std::regex_search(it, ed, m, rg)) {
        std::string fullMatch = m[0]; // 整段 src="xxx"
        std::string link = m[2].str(); // xxx
        // 轉成絕對網址
        std::string abs = dl->absoluteURL(baseUrl, link);
        // 變成本地檔名
        std::string local = urlToFilename(abs);
        // 如果連結是目錄(沒有檔名)就不要改
        if (local.empty()) {
            it = m[0].second;
            continue;
        }
        // 用本地檔名取代
        std::string replaced = m[1].str() + "\\s*" + local + "\"";
        out = std::regex_replace(out, std::regex(fullMatch), replaced);

        it = m[0].second;
    }

    return out;
}
// HttpDownloader 建構子
HttpDownloader::HttpDownloader(const std::string &startUrl,
                               const std::string &outDir,
                               const DownloadOptions &opt,
                               GlobalStats* stats)

```

```

: rootUrl(startUrl),
  outDir(outDir),
  opt(opt),
  stats(stats)
{
  string path;
  int port;
  // 解析成 rootScheme / rootHost (用來判斷外網)
  parseURL(startUrl, rootScheme, rootHost, port, path);
}
// parseURL :解析 URL
// 把 URL 分成 scheme(http), host(domain), port, path(/abc/xxx.htm)
bool HttpDownloader::parseURL(const std::string &url,
                               std::string &scheme,
                               std::string &host,
                               int &port,
                               std::string &path)
{
  scheme = "http";
  port   = 80;
  host.clear();
  path = "/";
  // 1. 找 "://"
  size_t p = url.find("://");
  size_t start = (p == string::npos ? 0 : p + 3);
  // 2. 找到第一個 '/'
  size_t slash = url.find('/', start);
  string hostPort;
  if (slash == string::npos) {
    //沒有 path 的情況
    hostPort = url.substr(start);
    path = "/";
  } else {
    hostPort = url.substr(start, slash - start);
    path = url.substr(slash);
  }
  // 看有沒有 port
  size_t colon = hostPort.find(':');
  if (colon == string::npos) {
    host = hostPort;
  } else {
    host = hostPort.substr(0, colon);
    port = stoi(hostPort.substr(colon + 1));
  }
  return !host.empty();
}

```

```

// isSameHost :判斷是否屬於內網(相同 host)
bool HttpDownloader::isSameHost(const std::string &url) {
    string s, h, path;
    int port;
    parseURL(url, s, h, port, path);
    return (h == rootHost);
}

// absoluteURL :把 link 變成絕對 URL
// 像是 /abc/cat.jpg 變成 http://host/abc/cat.jpg
string HttpDownloader::absoluteURL(const string &baseUrl,
                                   const string &link)
{
    if (link.empty()) return "";
    // 已經是絕對網址
    if (link.rfind("http://", 0) == 0 ||
        link.rfind("https://", 0) == 0)
        return link;

    string s, h, path;
    int port;
    parseURL(baseUrl, s, h, port, path);
    // 以根目錄開頭
    if (link[0] == '/') {
        return s + "://" + h + link;
    }
    // 否則在 baseUrl 所在的資料夾下組合
    size_t pos = path.find_last_of('/');
    string dir = (pos == string::npos ? "/" : path.substr(0, pos + 1));
    return s + "://" + h + dir + link;
}

// start:開始遞迴下載
void HttpDownloader::start(AdvancedDownloader *adv) {
    ensureDir(outDir); // 確保資料夾存在
    visited.clear(); // 清空以抓取紀錄(避免重複)
    crawl(rootUrl, 0, adv); // 從起始 URL 開始抓
}

// crawl (遞迴核心):主 HTML 遞迴
void HttpDownloader::crawl(const string &url,
                           int depth,
                           AdvancedDownloader *adv)
{
    if (!stats || stats->stopFlag) return;
    // 重複網址，不要抓
    if (visited.count(url)) return;
    visited.insert(url);
    // 不是 HTML，丟給進階多執行緒下載器(圖片、PDF)
    if (!isHtmlLike(url)) {

```

```

        if (adv) adv->addTask(url);
        return;
    }
    // 是HTML，抓內容（不寫檔）
    string html;
    if (!downloadPage(url, html)) return;
    // 抽取內部連結
    extractLinks(url, html, depth, adv);
    // 深度限制
    if (opt.maxDepth == 0) return;
    if (depth >= opt.maxDepth) return;
}
// extractLinks:負責從 HTML 找所有 link，遞迴爬下去
void HttpDownloader::extractLinks(const string &baseUrl,
                                   const string &html,
                                   int depth,
                                   AdvancedDownloader *adv)
{
    regex rg("href\\s*=\\s*\"([^\"]+)\"|src\\s*=\\s*\"([^\"]+)\"\"",
             regex::icase);
    smatch m;

    string::const_iterator it = html.begin();
    string::const_iterator ed = html.end();

    while (regex_search(it, ed, m, rg)) {
        string link = m[1].matched ? m[1].str() : m[2].str();
        string abs = absoluteURL(baseUrl, link);
        if (abs.empty()) {
            it = m[0].second;
            continue;
        }
        // 不允許外站
        if (!opt.allowExternal && !isSameHost(abs)) {
            it = m[0].second;
            continue;
        }
        bool htmlLike = isHtmlLike(abs);
        if (htmlLike) {
            // 遞迴下去
            if (opt.maxDepth > 0 && depth < opt.maxDepth)
                crawl(abs, depth + 1, adv);
        }
        else {
            // 加入下載任務(圖片等等)
            if (adv) adv->addTask(abs);
        }
    }
}

```

```

        it = m[0].second;
    }
}
// downloadPage:下載單一 HTML 本體 (只解析，不寫成檔案)
bool HttpDownloader::downloadPage(const std::string &url, std::string &body) {
    body.clear(); // HTML 內容先清空
    if (!stats) return false; // 沒有統計，不下載
    // 1. 用 parseURL 分解 URL -> scheme host port path
    string scheme, host, path;
    int port;
    if (!parseURL(url, scheme, host, port, path)) {
        cout << "[URL 錯誤] " << url << "\n";
        return false;
    }
    // 2. 第一次連線:只抓 header，取 Content-Length
    WSADATA wsa;
    WSASStartup(MAKEWORD(2,2), &wsa);

    addrinfo hints{}, *res;
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    string portStr = to_string(port);
    // DNS 查詢
    if (getaddrinfo(host.c_str(), portStr.c_str(), &hints, &res) != 0) {
        cout << "[連線失敗] " << host << "\n";
        WSACleanup();
        return false;
    }
    // 建立 socket + connect
    SOCKET s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
    if (connect(s, res->ai_addr, (int)res->ai_addrlen) == SOCKET_ERROR) {
        cout << "[連線失敗] " << host << "\n";
        freeaddrinfo(res);
        closesocket(s);
        WSACleanup();
        return false;
    }
    freeaddrinfo(res);
    // 發送 GET header
    string req =
        "GET " + path + " HTTP/1.1\r\n"
        "Host: " + host + "\r\n"
        "Connection: close\r\n\r\n";

    send(s, req.c_str(), (int)req.size(), 0);
    // 讀 header，找 Content-Length

```

```

string header;
char buf[4096];
long long remoteSize = -1; // 預設沒有給大小

while (true) {
    int n = recv(s, buf, sizeof(buf), 0);
    if (n <= 0) break;
    header.append(buf, n);
    size_t pos = header.find("\r\n\r\n");
    if (pos != string::npos) {

        string h = header.substr(0, pos + 4);

        regex cl("Content-Length:\\s*([0-9]+)", regex::icase);
        smatch m;
        if (regex_search(h, m, cl)) {
            remoteSize = stoll(m[1].str()); // 取得 HTML 大小
        }
        break;
    }
}
// 關閉第一次 socket
closesocket(s);
WSACleanup();

// 3. 第二次連線:正式下載(實際取) HTML body
// 同樣流程: DNS -> connect
WSAStartup(MAKEWORD(2, 2), &wsa);
addrinfo *res2;

if (getaddrinfo(host.c_str(), portStr.c_str(), &hints, &res2) != 0) {
    cout << "[連線失敗] " << host << "\n";
    WSACleanup();
    return false;
}

SOCKET s2 = socket(res2->ai_family, res2->ai_socktype, res2->ai_protocol);
if (connect(s2, res2->ai_addr, (int)res2->ai_addrlen) == SOCKET_ERROR) {
    cout << "[連線失敗] " << host << "\n";
    freeaddrinfo(res2);
    closesocket(s2);
    WSACleanup();
    return false;
}
freeaddrinfo(res2);
// 發送 GET
string req2 =

```

```

    "GET " + path + " HTTP/1.1\r\n"
    "Host: " + host + "\r\n"
    "Connection: close\r\n\r\n";
send(s2, req2.c_str(), (int)req2.size(), 0);
// 開始下載 HTML 主內容(body)
// headerDone = 0, 還在解析 header
// headerDone = 1, 開始把資料寫進 body
bool headerDone = false;
string recvHeader;
long long got = 0;
unsigned long long fileStart = GetTickCount64();

// 3_1. 決定要不要顯示 [HTML] 下載
// 如果 user 沒有限制副檔名, HTML 頁算是要下載的檔案
bool showHtmlLog = false;
if (opt.fileTypes.empty()) {
    // 沒有限制, HTML 也算要下載的, 顯示出來
    showHtmlLog = true;
} else {
    // 有限制, 只有使用者有填 html / htm 才會顯示
    for (auto t : opt.fileTypes) {
        for (char &c : t) c = tolower(c);
        if (t == "html" || t == "htm") {
            showHtmlLog = true;
            break;
        }
    }
}
if (showHtmlLog) {
    cout << "\n[HTML] 下載 " << url << "\n";
}
// 3_2. 下載迴圈(解析 header + 抓 HTML body)
while (true) {
    if (stats->stopFlag) break;
    // 支援 q 中止
    if (_kbhit()) {
        char c = _getch();
        if (c == 'q' || c == 'Q') {
            stats->stopFlag = true;
        }
    }
    int n = recv(s2, buf, sizeof(buf), 0);
    if (n <= 0) break;

    if (!headerDone) {
        // 拚 header
        recvHeader.append(buf, n);
    }
}

```

```

size_t pos = recvHeader.find("\r\n\r\n");
if (pos != string::npos) {
    int remain = n - (int)(pos + 4);
    // header 後面的資料 -> body
    if (remain > 0) {
        body.append(buf + pos + 4, remain);
        got += remain;
    }
    headerDone = true;
}
} else {
    // header 完成後，全部是 HTML body
    body.append(buf, n);
    got += n;
}
// 顯示 HTML 的下載進度(不影響檔案)
if (stats->showProgress) {
    unsigned long long now = GetTickCount64();
    double elapsed = (now - fileStart) / 1000.0;
    if (elapsed <= 0) elapsed = 0.001;

    double speedKB = got / 1024.0 / elapsed;

    int pct = 0;
    if (remoteSize > 0) {
        pct = (int)(100.0 * got / remoteSize);
        if (pct > 100) pct = 100;
    }

    double eta = 0;
    if (remoteSize > 0 && got > 0) {
        double left = (double)remoteSize - got;
        eta = left / (got / elapsed + 1e-6);
    }

    cout << "\r      "
         << setw(3) << pct << "%  "
         << fixed << setprecision(1) << speedKB << "KB/s  "
         << "ETA " << fixed << setprecision(1)
         << (eta > 0 ? eta : 0.0) << "s      " << flush;
}
}

// 關閉 socket
closesocket(s2);
WSACleanup();

if (stats->showProgress) cout << "\n";

```

```

// 4. 決定這個 HTML 要不要當成真正下載的檔案
// 訂定的規則：
// (1)沒有限制副檔名:HTML 也當成一般檔案
// (2)有限制，且包含 html / htm:HTML 也算
// (3)其他情況:HTML 只拿來解析連結，不存檔、不計數
bool needHtmlFile = false;
if (opt.fileTypes.empty()) {
    // 使用者沒有設定檔案類型，全部物件都要
    needHtmlFile = true;
} else {
    for (auto t : opt.fileTypes) {
        for (char &c : t) c = (char)tolower((unsigned char)c);
        if (t == "html" || t == "htm") {
            needHtmlFile = true;
            break;
        }
    }
}
// 如果這個 HTML 不需要當成檔案，就不要存檔、不更新統計
// 只保留在 body 字串裡讓上層去 extractLinks()
if (!needHtmlFile) {
    return !stats->stopFlag;
}
// 5. 真的需要 HTML 檔案寫成本地檔案(含 rewrite)
{
    std::string rewritten = rewriteHTML(body, url, this);
    // (1)產生檔名，將網址解析 host + path
    // 有 filename，用 urlToFile，沒有 filename (外網)，用 host_index.htm
    // 避免 .htm 被覆蓋
    std::string scheme, host, path;
    int port;
    parseURL(url, scheme, host, port, path);
    // 取得最後檔名
    std::string fname = urlToFilename(url);
    // 如果是資料夾 (沒有檔名)
    if (fname.empty()) {
        // 外網不能全部叫 index.htm，不然會彼此覆蓋
        // 所以加上 domain 作為前綴
        fname = host + "_index.htm";
    }
    // (2)過濾掉不合法字元
    for (char &c : fname) {
        if (c == '/' || c == '\\' || c == ':' ||
            c == '*' || c == '?' || c == '\"' ||
            c == '<' || c == '>' || c == '|' )
            c = '_';
    }
}

```

```

// (3)寫檔
std::string saveFile = outDir + "/" + fname;
std::ofstream fout(saveFile, std::ios::binary);
if (fout) {
    fout.write(rewritten.data(), (std::streamsize)rewritten.size());
}
}
// 6. 更新統計資訊(HTML 也算檔案)
{
    std::lock_guard<std::mutex> lock(stats->mtx);

    stats->totalBytes += got;
    stats->totalFiles += 1;

    double mb = stats->totalBytes / 1024.0 / 1024.0;
    double elapsedTotal =
        (GetTickCount64() - stats->startTick) / 1000.0;

    std::cout << "    [統計] 已完成檔案數: " << stats->totalFiles
        << "    已下載: " << std::fixed << std::setprecision(3) << mb << " MB"
        << "    總耗時: " << std::fixed << std::setprecision(1)
        << elapsedTotal << " 秒\n";
}

return !stats->stopFlag;
}

```

● advanced.h:

1. 特別撰寫的程式設計

(1) 重要部分

- AdvancedDownloader class 完全自行定義
- Task queue + taskSet 避免重複下載
- allowTypes(副檔名白名單)
- minSize(檔案大小下限)
- ThreadProgress 結構(你自己想的)
- monitor thread + worker threads 設計
- mutex 保護共享資料、monitorStop 旗標
- (2) 使用 std::queue + std::set 避免重複下載
- (3) 先用 HttpDownloader 遞迴抓 link，再交給 worker 真正下載
- (4) 完整多執行緒架構

2. advanced.h 完整 code & 註解

```

// 避免這個檔案被include多次
#pragma once

```

```

#include <string>
#include <vector>
#include <queue>

// 避免多執行緒衝突
#include <mutex>
// 用 set 避免網址重複加入
#include <set>
// 需要 DownloadOptions / GlobalStats
#include "downloader.h"

// 1. 每個 worker 的進度資訊(顯示用的)
struct ThreadProgress {
    int percent = 0; // 下載的百分比(0%~100%)
    double speedKB = 0.0; // 下載的速度(KB/s)
    double etaSec = 0.0; // 預估剩餘秒數
    bool active = false; // 這個 worker 目前有沒有在下载
};

// 2. 多執行緒下載器:用來抓圖片/PDF/其他更多檔案
class AdvancedDownloader {
public: // 建構子:指定最大執行緒數量、輸出之資料夾，等使用者設定
    AdvancedDownloader(int maxThreads,
                       const std::string &outDir,
                       const DownloadOptions &opt,
                       GlobalStats* stats);

    // 加入一下載任務(URL網址)，會放進 tasks queue 中
    void addTask(const std::string &url);

    // 限制只下載哪些副檔名(我的 code 是限制要小寫、也不要.，像是直接打 jpg)
    void setAllowTypes(const std::vector<std::string> &types);

    // 設定檔案大小下限(byte)，0 表示不會去限制
    void setMinSize(long long size);

    // 開始執行所有下載任務，會開 maxThreads 個 worker thread
    void run();

private:
    void workerThread(int id); // 每個 worker thread 會做的工作(負責下載一個檔案)
    void monitorLoop(int threads); // 監控 thread，定期印出所有多執行緒進度
private:
    int maxThreads; // 同時允許的執行緒數量
    std::string outDir; // 下載輸出的資料夾
    std::queue<std::string> tasks; // tasks queue(要下載的URL)
    std::mutex taskMutex; // tasks queue 的 mutex，避免多 thread 同時存取衝突

```

```

std::set<std::string> taskSet; // 避免重複加入相同 URL 進 tasks queue
std::vector<std::string> allowTypes; // 允許下載的副檔名清單
long long minSize; // 檔案霸小下限(byte)
bool allowAllTypes; // true=允許下載所有副檔名、false=有限制
DownloadOptions opt; // user 所有下載選項
GlobalStats* stats; // 全域統計的資料(下載量、檔案數量等等)
// 多线程 進度顯示相關
std::vector<ThreadProgress> progress; // 每個 worker 的進度
std::mutex progressMutex; // 保護 progress 的 mutex
bool monitorStop = false; // 是否要停止 monitor thread
};

```

● advanced.cpp:

1. 特別撰寫的程式設計

(1) workerThread()

- 負責取任務、判斷副檔名是否合法、處理續傳(Range 下載)、寫檔與更新紀錄、多執行緒安全統計(mutex)
- ETA (Estimated Time of Arrival)，是預估該 worker 正在下載的檔案還需要多少秒完成，其計算邏輯寫在 workerThread()，根據目前下載速度與剩餘 bytes 推算。

(2) monitorLoop()

- 自己寫的進度條邏輯(用 setw 清除到 120 字寬)
- 每 0.2 秒印出速度、ETA、進度百分比
- T1 / T2 / T3...: 代表每條多執行緒下載器 worker thread 的編號，每個 worker thread 專門負責下載非 HTML 檔案，這些列印是由 monitorLoop() 一樣是每 0.2 秒顯示一次。

2. advanced.cpp 完整 code & 註解

```

// AdvancedDownloader 的 class 定義
#include "advanced.h"
// urlToFilename() / ensureDir() 等工具
#include "util.h"

#include <iostream>
#include <fstream>
#include <filesystem>
#include <thread>
// setw(), setprecision()
#include <iomanip>
// Windows socket API
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <conio.h>
#pragma comment(lib, "ws2_32.lib")

```

```

using namespace std;
namespace fs = std::filesystem;

// getExtLower2():傳入路徑或是 URL 最後檔名，抽出副檔名並轉小寫
// 像是 image.JPG -> "jpg"
static std::string getExtLower2(const std::string &path) {
    // (1)去掉 query string (?abc=1)
    size_t q = path.find('?');
    std::string p = (q == std::string::npos ? path : path.substr(0, q));
    // (2)找副檔名最後一個'.'
    size_t pos = p.find_last_of('.');
    if (pos == std::string::npos) return ""; //沒副檔名回傳空字串
    // (3)抓副檔名並全部轉小寫
    std::string ext = p.substr(pos + 1);
    for (auto &c : ext) c = (char)tolower(c);
    return ext;
}

// 建構子:初始化成員之變數
AdvancedDownloader::AdvancedDownloader(int maxThreads,
                                       const std::string &outDir,
                                       const DownloadOptions &opt,
                                       GlobalStats* stats)
: maxThreads(maxThreads), // user 設定的最大執行緒數
  outDir(outDir), // 下載儲存的資料夾
  minSize(0), // 預設檔案下限 = 0 (也就是不限制)
  allowAllTypes(true), // 預設允許所有副檔名
  opt(opt), // 複製 user 設定
  stats(stats), // 全域統計的指標
  monitorStop(false) { // 監控執行緒停止的旗標
}

// 設定允許的副檔名
void AdvancedDownloader::setAllowTypes(const vector<string> &types) {
    allowTypes = types; // 儲存附檔名
    allowAllTypes = types.empty(); // 空的話，代表所有副檔名都允許
}

// 設定檔案下載的下限(byte)
void AdvancedDownloader::setMinSize(long long size) {
    minSize = size;
}

// addTask():新增一個要下載的URL
// 用 mutex 保護去避免多執行緒競爭，用 set 紀錄看過的 URL 去避免重複下載
void AdvancedDownloader::addTask(const string &url) {
    if (!stats) return; // 沒有統計的話不動
    if (stats->stopFlag) return; // 若 user 按 q，不再加入任務
    std::lock_guard<std::mutex> lock(taskMutex);
    // 避免重複下載同一個 URL
    if (taskSet.count(url)) {

```

```

        return;
    }
    taskSet.insert(url);
    tasks.push(url); // 加入下載 queue
}

// monitorLoop():負責顯示所有 worker thread 下載進度
// 每 0.2 秒更新一次(更新同一排文字，不會刷整個面)
void AdvancedDownloader::monitorLoop(int threads) {
    if (!stats) return;
    if (!stats->showProgress) return;
    while (!monitorStop && !stats->stopFlag) {
        {
            lock_guard<mutex> lock(progressMutex);
            // 先清掉當前那一整行文字
            cout << "\r";
            cout << setw(120) << " ";
            cout << "\r";

            cout << "進度:";
            // 印出每個 worker 的狀態
            for (int i = 0; i < threads; ++i) {
                if (i >= (int)progress.size()) break;
                cout << " T" << (i + 1) << " ";
                if (progress[i].active) { // 印出百分比、速度、ETA
                    cout << setw(3) << progress[i].percent << "% "
                        << fixed << setprecision(1) << progress[i].speedKB << "KB/s ";
                    cout << "ETA " << fixed << setprecision(1)
                        << (progress[i].etaSec > 0 ? progress[i].etaSec : 0.0) << "s ";
                } else {
                    cout << "-- ";
                }
            }
            cout << flush;
        }
        Sleep(200); // 每 0.2 秒更新一次
    }

    // 結束前清掉最後一行
    if (stats->showProgress) {
        cout << "\r" << setw(120) << " " << "\r" << flush;
    }
}

// run():負責啟動所有 worker thread & Monnitor thread
void AdvancedDownloader::run() {
    if (!stats) return;
    vector<thread> workers; // 存 worker thread
    int threads = maxThreads;

```

```

// 保護 thread 限制(最多=5)
if (threads < 1) threads = 1;
if (threads > 5) threads = 5;
// 初始化每個 worker 的進度
progress.assign(threads, ThreadProgress());
monitorStop = false;
// 若 user 有選擇顯示進度，開一個 monitor thread
thread monitor;
if (stats->showProgress) {
    monitor = thread(&AdvancedDownloader::monitorLoop, this, threads);
}
// 建立 worker threads (每個 thread 都會跑 workerThread(id))
for (int i = 0; i < threads; i++) {
    workers.emplace_back(&AdvancedDownloader::workerThread, this, i);
}
// 等所有 worker 結束
for (auto &t : workers) t.join();
// 通知 monitor thread 停止
monitorStop = true;
if (stats->showProgress && monitor.joinable()) {
    monitor.join();
}
}
// 每個 worker thread 都會跑這個函式，id = thread 編號
void AdvancedDownloader::workerThread(int id) {
    while (true) {
        // 如果按 q，全部的 worker 都要停止
        if (!stats) return;
        if (stats->stopFlag) return;
        string url;
        // 取下一個任務(critical section)
        {
            lock_guard<mutex> lock(taskMutex);
            //如果 queue 空了，這個 worker 就結束
            if (tasks.empty()) return;
            url = tasks.front(); // 取出最前面的 URL
            tasks.pop(); // 從 queue 移除
        }
        // 把 URL 轉成本地檔名
        string filename = urlToFilename(url);
        // urlToFilename 可能回傳""(例如是 html 或不合法的)，要略過
        if (filename.empty()) {
            continue; // 不要當成下載任務
        }
        string saveFile = outDir + "/" + filename;
        // 副檔名(全小寫)
        string ext = getExtLower2(filename);
    }
}

```

```

// 檔案名篩選
bool typeOK = false;
if (allowAllTypes) {
    typeOK = true; //若沒有限制副檔名，全部都允許
} else {
    for (auto &t : allowTypes) {
        if (ext == t) {
            typeOK = true;
            break;
        }
    }
}
if (!typeOK) {
    cout << "[略過類型] " << url << "\n";
    continue;
}
// 取得本地端已下載的大小(續傳用的)
long long localSize = 0;
if (fs::exists(saveFile))
    localSize = fs::file_size(saveFile);
// 第一次連線:只為了取 Content-Length (遠端檔案大小)
WSADATA wsa;
WSAStartup(MAKEWORD(2,2), &wsa);
string host, path;
int port = 80;
// 解析 URL :分割 host / path
size_t p = url.find("://");
string u = (p == string::npos ? url : url.substr(p+3));
size_t slash = u.find('/');
if (slash == string::npos) {
    host = u;
    path = "/";
} else {
    host = u.substr(0, slash);
    path = u.substr(slash);
}
// 支援 URL : port
string portStr = "80";
size_t colon = host.find(':');
if (colon != string::npos) {
    portStr = host.substr(colon+1);
    host = host.substr(0, colon);
}
// 域名轉 IP
struct addrinfo hints{}, *res;
hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;

```

```

if (getaddrinfo(host.c_str(), portStr.c_str(), &hints, &res) != 0) {
    cout << "[連線失敗]" << url << "\n";
    WSACleanup();
    continue;
}

SOCKET s = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
if (connect(s, res->ai_addr, (int)res->ai_addrlen) == SOCKET_ERROR) {
    cout << "[連線失敗]" << url << "\n";
    freeaddrinfo(res);
    closesocket(s);
    WSACleanup();
    continue;
}
freeaddrinfo(res);
// 發送 GET, 抓 header
string req = "GET " + path + " HTTP/1.1\r\n";
req += "Host: " + host + "\r\n";
req += "Connection: close\r\n\r\n";
send(s, req.c_str(), (int)req.size(), 0);
string header;
char buf[4096];
long long remoteSize = -1;
// 收 header
while (true) {
    int n = recv(s, buf, sizeof(buf), 0);
    if (n <= 0) break;
    header.append(buf, n);
    size_t pos = header.find("\r\n\r\n");
    if (pos != string::npos) {
        // 找 Content-Length
        string h = header.substr(0, pos+4);
        size_t cp = h.find("Content-Length:");
        if (cp != string::npos) {
            cp += 15;
            while (cp < h.size() && isspace((unsigned char)h[cp])) cp++;
            remoteSize = atoll(&h[cp]);
        }
        break;
    }
}

closesocket(s);
WSACleanup();

// 續傳 / 略過判斷

```

```

bool doResume = false;
// 續傳模式 (resumeMode = 1)，且小於 remoteSize，執行續傳
if (opt.resumeMode == 1 &&
    remoteSize > 0 &&
    localSize > 0 &&
    localSize < remoteSize) {
    doResume = true;
    cout << "[續傳檔案] " << url << "\n";
}
else if (opt.resumeMode == 1 &&
    remoteSize > 0 &&
    localSize >= remoteSize) {
    // 本地已經下載完成
    cout << "[略過已完成檔案] " << saveFile << "\n";
    continue;
}

// 第二次連線:開始正式下載檔案
WSAStartup(MAKEWORD(2,2), &wsa);

struct addrinfo *res2;
if (getaddrinfo(host.c_str(), portStr.c_str(), &hints, &res2) != 0) {
    cout << "[連線失敗] " << url << "\n";
    WSACleanup();
    continue;
}

SOCKET s2 = socket(res2->ai_family, res2->ai_socktype, res2->ai_protocol);
if (connect(s2, res2->ai_addr, (int)res2->ai_addrlen) == SOCKET_ERROR) {
    cout << "[連線失敗] " << url << "\n";
    freeaddrinfo(res2);
    closesocket(s2);
    WSACleanup();
    continue;
}
freeaddrinfo(res2);
string req2;
ofstream fout;
long long got = localSize; // 已經下載的大小(續傳用)
if (doResume) { // 有 Range 的續傳 GET
    req2 = "GET " + path + " HTTP/1.1\r\n";
    req2 += "Host: " + host + "\r\n";
    req2 += "Range: bytes=" + to_string(localSize) + "-\r\n";
    req2 += "Connection: close\r\n\r\n";
    fout.open(saveFile, ios::binary | ios::app);
} else { // 一般下載(從頭開始)
    req2 = "GET " + path + " HTTP/1.1\r\n";

```

```

    req2 += "Host: " + host + "\r\n";
    req2 += "Connection: close\r\n\r\n";
    fout.open(saveFile, ios::binary | ios::trunc);
    got = 0;
}

send(s2, req2.c_str(), (int)req2.size(), 0);
bool headerDone = false;
string recvHeader;
unsigned long long fileStart = GetTickCount64();

cout << "[檔案] 下載 " << url << endl;
// 下載主迴圈
while (true) {
    if (stats->stopFlag) break;

    // 按 q 停止，設 stopFlag
    if (_kbhit()) {
        char c = _getch();
        if (c == 'q' || c == 'Q') {
            stats->stopFlag = true;
        }
    }

    int n = recv(s2, buf, sizeof(buf), 0);
    if (n <= 0) break;
    // 處理header
    if (!headerDone) {
        recvHeader.append(buf, n);
        size_t pos = recvHeader.find("\r\n\r\n");
        if (pos != string::npos) {
            int remain = n - (int)(pos + 4);
            if (remain > 0) {
                // header 後面的資料寫進檔案
                fout.write(buf + pos + 4, remain);
                got += remain;
            }
            headerDone = true;
        }
    } else { // header 已完成，全部都是檔案內容
        fout.write(buf, n);
        got += n;
    }

    // 更新下載進度，由 monitorLoop 統一印一行
    if (stats->showProgress && remoteSize > 0) {
        unsigned long long now = GetTickCount64();

```

```

        double elapsed = (now - fileStart) / 1000.0;
        if (elapsed <= 0.0) elapsed = 0.001;
        double speedKB = (got - localSize) / 1024.0 / elapsed;

        int pct = (int)(100.0 * got / remoteSize);
        if (pct > 100) pct = 100;

        double eta = 0.0;
        if (got > 0) {
            double left = (double)remoteSize - got;
            eta = left / ((got - localSize) / elapsed + 1e-6);
        }

        lock_guard<mutex> lock(progressMutex);
        if (id < (int)progress.size()) {
            progress[id].active = true;
            progress[id].percent = pct;
            progress[id].speedKB = speedKB;
            progress[id].etaSec = eta;
        }
    }
}
fout.close();
closesocket(s2);
WSACleanup();
// 檔案下載完畢，標記 worker 空閒 (給 monitor thread 知道)
if (stats->showProgress) {
    lock_guard<mutex> lock(progressMutex);
    if (id < (int)progress.size()) {
        progress[id].active = false;
    }
}

// 檔案大小限制判斷 (下限)
if (minSize > 0 && got < minSize) {
    cout << "    [刪除] " << filename << " (" << got
        << " bytes) 小於下限 " << minSize << "\n";
    fs::remove(saveFile);
    continue;
}

// 更新全域統計(總檔案數、總 byte 數)
{
    std::lock_guard<std::mutex> lock(stats->mtx);
    stats->totalBytes += (got - localSize); // 新增下載的 bytes
    stats->totalFiles += 1; // 完成檔案 + 1
}

```

```

double mb = stats->totalBytes / 1024.0 / 1024.0;
double elapsedTotal = (GetTickCount64() - stats->startTick) / 1000.0;

cout << "\n";
cout << "    [統計] 已完成檔案數: " << stats->totalFiles
    << "    已下載: " << fixed << setprecision(3) << mb << " MB"
    << "    總耗時: " << fixed << setprecision(1)
    << elapsedTotal << " 秒\n";
}
}
}

```

● main.cpp:

1. 特別撰寫的程式設計

(1) 全部邏輯

- 命令列模式解析參數
- 互動模式詢問使用者
- 字串輸入與 error 過濾
- 使用 ensureDir 建立資料夾
- 顯示最終統計
- 跑 dl.start() + adv.run() 的整體流程控制
- 支援 minSize / allowTypes 的整合
- 支援中止後詢問是否繼續下載

(2) Win10 UTF-8 輸出支援 (SetConsoleOutputCP)

2. main.cpp 完整 code & 註解

```

#include <iostream>
#include <filesystem>
#include <vector>
#include <string>
// Windows API 設定 UTF-8 編碼、計時
#include <windows.h>

// HTML 下載器，負責爬蟲
#include "downloader.h"
// 進階下載器，負責多 thread 下載(圖片 / PDF / 其他檔案)
#include "advanced.h"
// 工具(urlToFilename / ensureDir)
#include "util.h"

using namespace std;
// 避免每次都要打 std::filesystem，用 fs 代替
namespace fs = std::filesystem;

// 1. 判斷字串是不是數字，用來判斷 size

```

```

bool isNumber(const std::string &s) {
    if (s.empty()) return false; // 空字串不是數字
    for (char c : s)
        // 只要有不是數字的字元就回傳 false
        if (!isdigit((unsigned char)c)) return false;
    return true;
}

// 2. 刪除整個下載資料夾資料
void clearDownloadFolder(const std::string &outDir) {
    if (fs::exists(outDir)) { // 會先檢查資料夾存不存在
        cout << ">> 刪除舊的下載資料夾: " << outDir << "\n";
        fs::remove_all(outDir); // 刪除整個資料夾及底下所有檔案
    }
}

int main(int argc, char* argv[]) {
    //設定 Windows Console 使用 UTF-8 , 避免亂碼
    SetConsoleOutputCP(CP_UTF8); // cout 為 UTF-8
    SetConsoleCP(CP_UTF8); // cin 為 UTF-8

    // 3. 命令列模式 - 一次性輸入模式
    if (argc >= 8) { // 至少要8參數才能進入命令列模式
        string url = argv[1]; // 起始 URL
        string outDir = argv[2]; // 輸出資料夾名稱
        int depth = atoi(argv[3]); // 遞迴深度
        bool allowExt = (argv[4][0] == 'y' || argv[4][0] == 'Y'); // 允許外部網站
        bool showProgress = (argv[5][0] == 'y' || argv[5][0] == 'Y'); // 顯示進度
        int resumeMode = atoi(argv[6]); // 下載模式
        int maxThreads = atoi(argv[7]); // 同時下載檔案數上限
        vector<string> allowTypes; // user 限制要下載的副檔名
        long long minSize = 0; // user 限制檔案大小下限

        // 後面參數:若是數字視為 minSize, 否則視為副檔名
        for (int i = 8; i < argc; ++i) {
            string arg = argv[i];
            if (isNumber(arg) && minSize == 0) {
                minSize = atoll(arg.c_str()); // 轉成 long long
            } else {
                allowTypes.push_back(arg); // 副檔名加入 vector
            }
        }

        // 準備 DownloadOptions 結構(user 設定的所有下載選項)
        // 要傳給 HttpDownloader / AdvancedDownloader 使用
        DownloadOptions opt;
        opt.maxDepth = depth;
        opt.allowExternal = allowExt;
        opt.resumeMode = resumeMode;
        opt.showProgress = showProgress ? 1 : 0;
    }
}

```

```

opt.fileTypes = allowTypes;
// 若選擇刪除後重抓，先刪除舊資料夾
if (resumeMode == 2) clearDownloadFolder(outDir);
// 若資料夾不存在就建立
ensureDir(outDir);
// 建立 GlobalStats 結構(全域統計資訊)
// 紀錄下載量、檔案數量等等
GlobalStats stats;
stats.totalBytes = 0;
stats.totalFiles = 0;
stats.startTick = GetTickCount64(); // 下載開始時間
stats.showProgress = showProgress; // 是否顯示進度
stats.stopFlag = false; // 是否停止下載(按q)

cout << "\n----- {命令列模式} -----\n";
//建立下載器物件
opt.fileTypes = allowTypes;
AdvancedDownloader adv(maxThreads, outDir, opt, &stats);
if (!allowTypes.empty()) adv.setAllowTypes(allowTypes);
if (minSize > 0)      adv.setMinSize(minSize);

HttpDownloader dl(url, outDir, opt, &stats);
// 主要下載流程(HTML -> 檔案 -> 檔案 ..... )
while (true) {
    dl.start(&adv); // 先下載 HTML + 解析連結
    adv.run(); // 再由多 thread 下載所有檔案

    if (!stats.stopFlag) break; // 沒被 q 中止，就跳出迴圈結束下載

    // 被 q 中止
    cout << "\n偵測到下載中止，要繼續下載嗎?(y/n): ";
    char c;
    cin >> c;
    if (c == 'y' || c == 'Y') {
        stats.stopFlag = false;
        cout << "\n== 繼續下載 ==\n";
        continue; // 再進行下一輪下載
    } else {
        cout << "\n== 使用者選擇結束下載 ==\n";
        break;
    }
}
// 顯示下載統計結果
double mb = stats.totalBytes / 1024.0 / 1024.0;
double elapsedTotal = (GetTickCount64() - stats.startTick) / 1000.0;

cout << "\n===== 下載流程結束 =====\n";

```

```

    cout << "總共完成檔案數: " << stats.totalFiles
        << " 總下載量: " << fixed << setprecision(3) << mb << " MB"
        << " 總耗時: " << fixed << setprecision(1)
        << elapsedTotal << " 秒\n";

    return 0;
}

// 互動模式
cout << "\n-----\n";
cout << "                {網路下載器 - 互動模式}\n";
cout << "-----\n\n";
string url, outDir;
DownloadOptions opt;

cout << "[基本設定]\n";
cout << " 請輸入 URL: ";
cin >> url;

cout << " 輸出資料夾名稱: ";
cin >> outDir;

cout << " 遞迴下載深度 (0 = 只抓起始頁): ";
cin >> opt.maxDepth;

char ch;

cout << "\n[網域設定]\n";
cout << " 允許外部網站? (y/n): ";
cin >> ch;
opt.allowExternal = (ch == 'y' || ch == 'Y');

cout << "\n[續傳 / 重抓]\n";
cout << " 下載模式 (0=正常 1=續傳 2=刪除後重抓): ";
cin >> opt.resumeMode;
// 重抓，刪除目標舊資料夾
if (opt.resumeMode == 2) clearDownloadFolder(outDir);
// 存不存在就建立資料夾
ensureDir(outDir);

cout << "\n[顯示選項]\n";
cout << " 顯示下載進度與統計? (y/n): ";
cin >> ch;
bool showProgress = (ch == 'y' || ch == 'Y');
opt.showProgress = showProgress ? 1 : 0;

int maxThreads;

```

```

cout << "\n[多執行緒]\n";
cout << " 同時下載檔案數上限 (1~5): ";
cin >> maxThreads;

vector<string> allowTypes;
long long minSize = 0;

cout << "\n[檔案限制]\n";
cout << " 只下載特定檔案類型? (y/n): ";
cin >> ch;
if (ch == 'y' || ch == 'Y') {
    cout << " 請輸入副檔名 (例如: jpg png pdf) , 以空白分隔: ";
    string ext;
    while (cin >> ext) {
        allowTypes.push_back(ext);
        if (cin.peek() == '\n') break;
    }
}

cout << " 設定檔案大小下限? (y/n): ";
cin >> ch;
if (ch == 'y' || ch == 'Y') {
    cout << " 最小大小 (byte): ";
    cin >> minSize;
}
// 初始化 GlobalStats (儲存所有下載統計資訊)
GlobalStats stats;
stats.totalBytes = 0;
stats.totalFiles = 0;
stats.startTick = GetTickCount64();
stats.showProgress = showProgress;
stats.stopFlag = false;

cout << "\n===== 下載流程開始 =====\n";
// 建立下載物件 (HTML 下載器 + 進階多執行緒下載器)
opt.fileTypes = allowTypes;
AdvancedDownloader adv(maxThreads, outDir, opt, &stats);
if (!allowTypes.empty()) adv.setAllowTypes(allowTypes);
if (minSize > 0) adv.setMinSize(minSize);

HttpDownloader dl(url, outDir, opt, &stats);
// 主要下載流程: HTML -> 多執行緒下載
while (true) {
    dl.start(&adv); // 下載 HTML 並解析連結
    adv.run(); // 多執行緒下載所有檔案

    if (!stats.stopFlag) break; // 若沒有按 q , 跳出迴圈結束下載
}

```

```

cout << "\n偵測到下載中止，要繼續下載嗎？(y/n): ";
cin >> ch;
if (ch == 'y' || ch == 'Y') {
    stats.stopFlag = false;
    cout << "\n== 繼續下載 ==\n";
    continue;
} else {
    cout << "\n== 使用者選擇結束下載 ==\n";
    break;
}
}
// 計算並輸出最後下載統計結果
double mb = stats.totalBytes / 1024.0 / 1024.0;
double elapsedTotal = (GetTickCount64() - stats.startTick) / 1000.0;

cout << "\n===== 下載流程結束 =====\n";
cout << "總共完成檔案數: " << stats.totalFiles
    << " 總下載量: " << fixed << setprecision(3) << mb << " MB"
    << " 總耗時: " << fixed << setprecision(1)
    << elapsedTotal << " 秒\n";

return 0;
}

```

三、程式操作說明

- 本專題操作撰寫環境是在 VS code 中(所有操作都在 VS code 中的 cmd

1. 所有程式檔案都準備好後，在 cmd 輸入編譯指令：

```
g++ -std=c++17 main.cpp downloader.cpp advanced.cpp -lws2_32 -o web.exe
```

將所有程式一併編譯，編譯後會有 web.exe 出現，如下圖。



2. 模式一:互動模式，可直接在 cmd 中執行: web，即會進入互動模式，逐一填入需求參數即可，如下所示。

格式：

[基本設定]

請輸入 URL: `http://hsccl.fr.to/index.htm`
輸出資料夾名稱: `test_1`
遞迴下載深度 (0 = 只抓起始頁): 3

[網域設定]

允許外部網站? (y/n): n

[續傳 / 重抓]

下載模式 (0=正常 1=續傳 2=刪除後重抓): 2
>> 刪除舊的下載資料夾: `test_1`

[顯示選項]

顯示下載進度與統計? (y/n): n

[多執行緒]

同時下載檔案數上限 (1~5): 1

[檔案篩選]

只下載特定檔案類型? (y/n): y
請輸入副檔名 (例如: `jpg png pdf`) , 以空白分隔: `png`
設定檔案大小下限? (y/n): y
最小大小 (byte): 200000

```
----- (網路下載器 - 互動模式) -----

[基本設定]
請輸入 URL: http://hsccl.fr.to/index.htm
輸出資料夾名稱: test_1
遞迴下載深度 (0 = 只抓起始頁): 3

[網域設定]
允許外部網站? (y/n): n

[續傳 / 重抓]
下載模式 (0=正常 1=續傳 2=刪除後重抓): 2
>> 刪除舊的下載資料夾: test_1

[顯示選項]
顯示下載進度與統計? (y/n): 1

[多執行緒]
同時下載檔案數上限 (1~5): 1

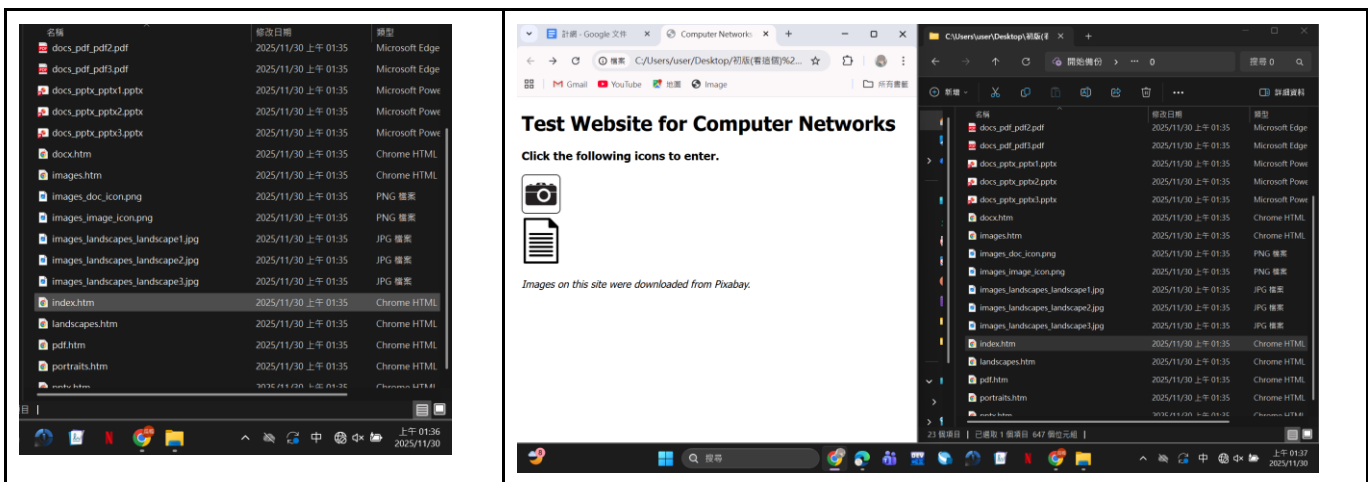
[檔案限制]
只下載特定檔案類型? (y/n): y
請輸入副檔名 (例如: jpg png pdf) , 以空白分隔: png
設定檔案大小下限? (y/n): y
最小大小 (byte) : 200000
===== 下載流程開始 =====
[略過類型] http://hsccl.fr.to/images/animals/cats/cat1.jpg
[略過類型] http://hsccl.fr.to/images/animals/cats/cat2.jpg
[略過類型] http://hsccl.fr.to/images/animals/cats/cat3.jpg
[略過類型] http://hsccl.fr.to/images/animals/dogs/dog1.jpg
[略過類型] http://hsccl.fr.to/images/animals/dogs/dog2.jpg
[略過類型] http://hsccl.fr.to/images/animals/dogs/dog3.jpg
[略過類型] http://hsccl.fr.to/images/animals/rabbits/rabbit1.jpg
[略過類型] http://hsccl.fr.to/images/animals/rabbits/rabbit2.jpg
[略過類型] http://hsccl.fr.to/images/animals/rabbits/rabbit3.jpg
[略過類型] http://hsccl.fr.to/images/portraits/kids/kid1.jpg
[略過類型] http://hsccl.fr.to/images/portraits/kids/kid2.jpg
[略過類型] http://hsccl.fr.to/images/portraits/kids/kid3.jpg
[略過類型] http://hsccl.fr.to/images/landscapes/landscape1.jpg
[略過類型] http://hsccl.fr.to/images/landscapes/landscape2.jpg
[略過類型] http://hsccl.fr.to/images/landscapes/landscape3.jpg
[檔案] 下載 http://hsccl.fr.to/images/image_icon.png
[刪除] images_image_icon.png (85162 bytes) 小於下限 200000
[略過類型] http://hsccl.fr.to/docs/docx/word1.docx
[略過類型] http://hsccl.fr.to/docs/docx/word2.docx
[略過類型] http://hsccl.fr.to/docs/docx/word3.docx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx1.pptx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx2.pptx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx3.pptx
[略過類型] http://hsccl.fr.to/docs/pdf/pdf1.pdf
[略過類型] http://hsccl.fr.to/docs/pdf/pdf2.pdf
[略過類型] http://hsccl.fr.to/docs/pdf/pdf3.pdf
[檔案] 下載 http://hsccl.fr.to/images/doc_icon.png
[刪除] images_doc_icon.png (31080 bytes) 小於下限 200000
===== 下載流程結束 =====
總共完成檔案數: 0 總下載量: 0.000 MB 總耗時: 1.2 秒
```

3. 模式二:命令列模式，直接照著互動式格式順序，依次打在同一行指令輸出即可，如下所示。
格式：

```
web.exe http://hsccl.fr.to/index.htm test_2 3 n n 0 1 200000 jpg
```

```
C:\Users\user\Desktop\初版(舊地圖) - 成功外網 - 複製web.exe http://hsccl.fr.to/index.htm test_2 3 n n 0 1 200000 jpg
----- (命令列模式) -----
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat1.jpg
[統計] 已完成檔案數: 1 已下載: 0.373 MB 總耗時: 1.1 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat2.jpg
[統計] 已完成檔案數: 2 已下載: 0.583 MB 總耗時: 1.2 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat3.jpg
[統計] 已完成檔案數: 3 已下載: 0.844 MB 總耗時: 1.3 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog1.jpg
[統計] 已完成檔案數: 4 已下載: 1.072 MB 總耗時: 1.4 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog2.jpg
[統計] 已完成檔案數: 5 已下載: 1.334 MB 總耗時: 1.5 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog3.jpg
[統計] 已完成檔案數: 6 已下載: 1.576 MB 總耗時: 1.6 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit1.jpg
[刪除] images\animals\rabbits_rabbit1.jpg (187396 bytes) 小於下限 200000
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit2.jpg
[統計] 已完成檔案數: 7 已下載: 1.850 MB 總耗時: 1.9 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit3.jpg
[統計] 已完成檔案數: 8 已下載: 2.060 MB 總耗時: 2.0 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid1.jpg
[統計] 已完成檔案數: 9 已下載: 2.383 MB 總耗時: 2.1 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid2.jpg
[統計] 已完成檔案數: 10 已下載: 2.558 MB 總耗時: 2.2 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid3.jpg
[統計] 已完成檔案數: 11 已下載: 2.907 MB 總耗時: 2.3 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape1.jpg
[統計] 已完成檔案數: 12 已下載: 3.624 MB 總耗時: 2.4 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape2.jpg
[統計] 已完成檔案數: 13 已下載: 3.822 MB 總耗時: 2.5 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape3.jpg
[統計] 已完成檔案數: 14 已下載: 4.233 MB 總耗時: 2.6 秒
[略過類型] http://hsccl.fr.to/images/image_icon.png
[略過類型] http://hsccl.fr.to/docs/docx/word1.docx
[略過類型] http://hsccl.fr.to/docs/docx/word2.docx
[略過類型] http://hsccl.fr.to/docs/docx/word3.docx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx1.pptx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx2.pptx
[略過類型] http://hsccl.fr.to/docs/pptx/pptx3.pptx
[略過類型] http://hsccl.fr.to/docs/pdf/pdf1.pdf
[略過類型] http://hsccl.fr.to/docs/pdf/pdf2.pdf
[略過類型] http://hsccl.fr.to/docs/pdf/pdf3.pdf
[略過類型] http://hsccl.fr.to/images/doc_icon.png
===== 下載流程結束 =====
總共完成檔案數: 14 總下載量: 4.233 MB 總耗時: 2.6 秒
```

4. 然後本程式有做到可離線瀏覽，只要打開資料夾，點選 .htm 即可看到抓下來的網站，其中的 png、jpg、pdf、ppt、word也都是可以=離線下載、瀏覽的，如下圖所示。



四、成果(兩個測試)

- 測試一：選擇互動模式，遞迴深度 2，多執行緒 = 4，顯示進度的話：

```
(網路下載器 - 互動模式)

[基本設定]
請輸入 URL: http://hscc1.fr.to/index.htm
輸出資料夾名稱: 3
遞迴下載深度 (0 = 只抓起始頁): 2

[網域設定]
允許外部網站? (y/n): n

[選擇 / 重組]
下載模式 (0=正常 1=續傳 2=刪除後重組): 2
>> 刪除舊的下載資料夾: 3

[顯示進度]
顯示下載進度與統計? (y/n): y

[多執行緒]
同時下載檔案數上限 (3-5): 4

[檔案限制]
只下載特定檔案類型? (y/n): n
設定檔案大小下限? (y/n): n
\===== 下載進度開始 =====

[HTML] 下載 http://hscc1.fr.to/index.htm
100% 633.8KB/s ETA 0.0s
[統計] 已完成檔案數: 1 已下載: 0.001 MB 總耗時: 0.2 秒

[HTML] 下載 http://hscc1.fr.to/images.htm
100% 33.2KB/s ETA 0.0s
[統計] 已完成檔案數: 2 已下載: 0.001 MB 總耗時: 0.3 秒

[HTML] 下載 http://hscc1.fr.to/animals.htm
100% 30.8KB/s ETA 0.0s
[統計] 已完成檔案數: 3 已下載: 0.002 MB 總耗時: 0.4 秒

[HTML] 下載 http://hscc1.fr.to/portraits.htm
100% 28.8KB/s ETA 0.0s
[統計] 已完成檔案數: 4 已下載: 0.002 MB 總耗時: 0.4 秒

[HTML] 下載 http://hscc1.fr.to/landscapes.htm
100% 705.1KB/s ETA 0.0s
[統計] 已完成檔案數: 5 已下載: 0.003 MB 總耗時: 0.5 秒

[HTML] 下載 http://hscc1.fr.to/docs.htm
100% 25.9KB/s ETA 0.0s
[統計] 已完成檔案數: 6 已下載: 0.003 MB 總耗時: 0.5 秒

[HTML] 下載 http://hscc1.fr.to/docs.htm
100% 33.1KB/s ETA 0.0s
[統計] 已完成檔案數: 7 已下載: 0.004 MB 總耗時: 0.6 秒

[HTML] 下載 http://hscc1.fr.to/pptx.htm
100% 34.5KB/s ETA 0.0s
[統計] 已完成檔案數: 8 已下載: 0.004 MB 總耗時: 0.6 秒

[HTML] 下載 http://hscc1.fr.to/pdf.htm
100% 34.4KB/s ETA 0.0s
[統計] 已完成檔案數: 9 已下載: 0.005 MB 總耗時: 0.7 秒
進度: T1 -- T2 -- T3 -- T4 -- [檔案] 下載 http://hscc1.fr.to/images/landscapes/landscape2.jpg [檔案] 下載
http://hscc1.fr.to/images/image_icon.png http://hscc1.fr.to/images/landscapes/landscape3.jpg

[統計] 已完成檔案數: 10 已下載: 0.202 MB 總耗時: 0.8 秒
[統計] 已完成檔案數: 11 已下載: 0.284 MB 總耗時: 0.8 秒
[統計] 已完成檔案數: 12 已下載: 0.695 MB 總耗時: 0.8 秒
[檔案] 下載 http://hscc1.fr.to/docs/docx/word1.docx
[檔案] 下載 http://hscc1.fr.to/docs/docx/word2.docx
[統計] 已完成檔案數: 13 已下載: 0.706 MB 總耗時: 0.9 秒
[檔案] 下載 http://hscc1.fr.to/docs/docx/word3.docx
[統計] 已完成檔案數: 14 已下載: 0.718 MB 總耗時: 0.9 秒
[統計] 已完成檔案數: 15 已下載: 0.729 MB 總耗時: 0.9 秒
進度: T1 -- T2 -- T3 -- T4 -- [檔案] 下載 http://hscc1.fr.to/docs/pptx/pptx1.pptx
[檔案] 下載 http://hscc1.fr.to/docs/pptx/pptx2.pptx
[檔案] 下載 http://hscc1.fr.to/docs/pptx/pptx3.pptx
[統計] 已完成檔案數: 16 已下載: 0.761 MB 總耗時: 1.0 秒
[檔案] 下載 http://hscc1.fr.to/images/landscapes/landscape1.jpg
[統計] 已完成檔案數: 17 已下載: 0.793 MB 總耗時: 1.0 秒
[統計] 已完成檔案數: 18 已下載: 0.826 MB 總耗時: 1.0 秒
[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf1.pdf
[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf2.pdf
[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf3.pdf
進度: T1 14% 716.3KB/s ETA 0.4s T2 9% 928.8KB/s ETA 0.1s T3 87% 5864.8KB/s ETA 0.0s T4 28% 1299.6KB/s ETA 0.2s
[統計] 已完成檔案數: 19 已下載: 1.543 MB 總耗時: 1.1 秒
[統計] 已完成檔案數: 20 已下載: 1.690 MB 總耗時: 1.2 秒
[檔案] 下載 http://hscc1.fr.to/images/doc_icon.png
[統計] 已完成檔案數: 21 已下載: 1.963 MB 總耗時: 1.2 秒
[統計] 已完成檔案數: 22 已下載: 1.993 MB 總耗時: 1.2 秒
[統計] 已完成檔案數: 23 已下載: 2.297 MB 總耗時: 1.2 秒
===== 下載進度結束 =====
總共完成檔案數: 23 總下載量: 2.297 MB 總耗時: 1.3 秒
```

(1)[HTML]: 表示這個是 HTML 頁面，由 HttpDownloader 下載。

(2)100%：這一個 HTML 檔下載完成。

(3)633.8KB/s：這一個 HTML 的下載速度。

(4)ETA 0.0s：預估剩餘時間（很快抓完所以是 0）。

(5)[統計]:是全程統計

(6)[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf1.pdf

(7)[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf2.pdf

(8)[檔案] 下載 http://hscc1.fr.to/docs/pdf/pdf3.pdf

進度：T1 14% 716.3KB/s ETA 0.4s T2 9% 928.8KB/s ETA 0.1s

T3 87% 5864.8KB/s ETA 0.0s T4 28% 1299.6KB/s ETA 0.2s

=>這邊的話我的 % 程式是：int pct = (int)(100.0 * got / remoteSize);

=>其中int pct = (int)(100.0 * got / remoteSize);

got = 目前已下載的 bytes

=>那 T3 很快就 87% 的原因是因為 T3 下載的檔案比較小或 T3 的檔案早一點被排給它，亦或是檔案可能在伺服器端比較快（快取）。

⇒ 多執行緒的進度列（T1~T4）只會顯示正在下載的執行緒，因此空閒中的 thread 顯示為 -- 屬正常行為。百分比的計算方式為：已下載位元組（got）除以 Content-Length（remoteSize），因此顯示的 % 是正確且精準的，因為測試網站的檔案都很小，加上使用 4 個 thread 同時下載，因此有些 thread 的進度會瞬間跳到較高的百分比，或很快顯示 ETA=0

● 測試二:選擇的是命令列模式，遞迴深度是3，多執行緒是1，不顯示進度，限制檔案類型為 “jpg”，檔案大小最小限制是 200000-byte (= 約200 KB):

```
C:\Users\user\Desktop\初版(舊版) - 成功外網 - 複製>web.exe http://hsccl.fr.to/index.htm 3 3 n n 2 1 200000 jpg
>> 刪除舊的下載資料夾: 3

----- (命令列模式) -----
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat1.jpg
[統計] 已完成檔案數: 1 已下載: 0.373 MB 總耗時: 1.1 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat2.jpg
[統計] 已完成檔案數: 2 已下載: 0.583 MB 總耗時: 1.2 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/cats/cat3.jpg
[統計] 已完成檔案數: 3 已下載: 0.844 MB 總耗時: 1.3 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog1.jpg
[統計] 已完成檔案數: 4 已下載: 1.072 MB 總耗時: 1.4 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog2.jpg
[統計] 已完成檔案數: 5 已下載: 1.334 MB 總耗時: 1.7 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/dogs/dog3.jpg
[統計] 已完成檔案數: 6 已下載: 1.576 MB 總耗時: 1.8 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit1.jpg
[刪除] images_animals_rabbits_rabbit1.jpg (187396 bytes) 小於下限 200000
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit2.jpg
[統計] 已完成檔案數: 7 已下載: 1.850 MB 總耗時: 2.0 秒
[檔案] 下載 http://hsccl.fr.to/images/animals/rabbits/rabbit3.jpg
[統計] 已完成檔案數: 8 已下載: 2.060 MB 總耗時: 2.1 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid1.jpg
[統計] 已完成檔案數: 9 已下載: 2.303 MB 總耗時: 2.2 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid2.jpg
[統計] 已完成檔案數: 10 已下載: 2.558 MB 總耗時: 2.3 秒
[檔案] 下載 http://hsccl.fr.to/images/portraits/kids/kid3.jpg
[統計] 已完成檔案數: 11 已下載: 2.907 MB 總耗時: 2.4 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape1.jpg
[統計] 已完成檔案數: 12 已下載: 3.624 MB 總耗時: 2.6 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape2.jpg
[統計] 已完成檔案數: 13 已下載: 3.822 MB 總耗時: 2.7 秒
[檔案] 下載 http://hsccl.fr.to/images/landscapes/landscape3.jpg
[統計] 已完成檔案數: 14 已下載: 4.233 MB 總耗時: 2.8 秒
[略過類型] http://hsccl.fr.to/images/image_icon.png
[略過類型] http://hsccl.fr.to/docs/docs/word1.docx
[略過類型] http://hsccl.fr.to/docs/docs/word2.docx
[略過類型] http://hsccl.fr.to/docs/docs/word3.docx
[略過類型] http://hsccl.fr.to/docs/ppts/pptx1.pptx
[略過類型] http://hsccl.fr.to/docs/ppts/pptx2.pptx
[略過類型] http://hsccl.fr.to/docs/ppts/pptx3.pptx
[略過類型] http://hsccl.fr.to/docs/pdfs/pdf1.pdf
[略過類型] http://hsccl.fr.to/docs/pdfs/pdf2.pdf
[略過類型] http://hsccl.fr.to/docs/pdfs/pdf3.pdf
[略過類型] http://hsccl.fr.to/images/doc_icon.png

***** 下載流程結束 *****
總共完成檔案數: 14 總下載量: 4.233 MB 總耗時: 2.8 秒
```

	images_animals_cats_...	2025/11/30 下午 12:20	JPG 檔案	383 KB
	images_animals_cats_...	2025/11/30 下午 12:20	JPG 檔案	215 KB
	images_animals_cats_...	2025/11/30 下午 12:20	JPG 檔案	268 KB
	images_animals_dogs...	2025/11/30 下午 12:20	JPG 檔案	234 KB
	images_animals_dogs...	2025/11/30 下午 12:20	JPG 檔案	268 KB
	images_animals_dogs...	2025/11/30 下午 12:20	JPG 檔案	248 KB
	images_animals_rabbi...	2025/11/30 下午 12:20	JPG 檔案	282 KB
	images_animals_rabbi...	2025/11/30 下午 12:20	JPG 檔案	215 KB
	images_landscapes_la...	2025/11/30 下午 12:20	JPG 檔案	735 KB
	images_landscapes_la...	2025/11/30 下午 12:20	JPG 檔案	203 KB
	images_landscapes_la...	2025/11/30 下午 12:20	JPG 檔案	422 KB
	images_portraits_kids...	2025/11/30 下午 12:20	JPG 檔案	249 KB
	images_portraits_kids...	2025/11/30 下午 12:20	JPG 檔案	262 KB
	images_portraits_kids...	2025/11/30 下午 12:20	JPG 檔案	358 KB

(1) 因為有限制檔案類型為 “jpg”，且大小不能小於約 200 KB，所以有得右邊照片的結果(只有下載這些檔案，其他檔案沒有被下載下來)

五、程式撰寫參考資訊

1. 因為作業邀求不可以用現成的 HTTP Library，所以我用我的筆電環境，Windows 的 Winsock 去實作 TCP socket 連線。

參考來源為微軟官方文件：<https://learn.microsoft.com/en-us/windows/win32/api/winsock2/>。

在程式有用到的對應部分(程式對應官方文件中的內容)：

函式	用途	有出現的程式模組
WSAStartup()	初始化 Winsock	downloader.cpp, advanced.cpp
socket()	建立 TCP 連線	downloader.cpp
connect()	連線 HTTP server	downloader.cpp
send()	傳送 HTTP GET 請求	downloader.cpp
recv()	讀取 server 回傳的 header / body	downloader.cpp
closesocket()	關閉 socket	downloader.cpp
WSACleanup()	清理 Winsock	downloader.cpp

2. TCP Client我有參考微軟提供的最基本 TCP client 範例，參考部分對應我的程式使用之 socket 建立、connect、send、recv。

參考來源為微軟官方範例：<https://learn.microsoft.com/en-us/windows/win32/winsock/complete-client-code>

我是參考其概念，參考到的概念：

(1) Winsock 初始化方式

微軟程式區段	我的程式區段
<pre>WSADATA wsaData; iResult = WSAStartup(MAKEWORD(2,2), &wsaData);</pre>	<pre>WSADATA wsa; WSAStartup(MAKEWORD(2,2), &wsa);</pre>

(2) 建立socket()、與server建立連線connect()，socket -> connect -> send -> recv 之流程

微軟程式區段	我的程式區段
<pre>iResult = connect(ConnectSocket, result->ai_addr, (int)result->ai_addrlen);</pre>	<pre>if (connect(s, res->ai_addr, (int)res->ai_addrlen) == SOCKET_ERROR) { cout << "[連線失敗]" << url << "\n"; closesocket(s); WSACleanup(); continue; }</pre>

iResult = send(ConnectSocket, sendbuf, (int)strlen(sendbuf), 0);	string req = "GET " + path + " HTTP/1.1\r\n"; req += "Host: " + host + "\r\n"; req += "Connection: close\r\n\r\n"; send(s, req.c_str(), (int)req.size(), 0);
iResult = recv(ConnectSocket, recvbuf, recvbuflen, 0);	int n = recv(s, buf, sizeof(buf), 0); if (n <= 0) break; header.append(buf, n);

(3)關閉 socket

微軟程式區段	我的程式區段
closesocket(ConnectSocket); WSACleanup();	closesocket(s); WSACleanup();

3. 在std::regex 抽取 HTML 連結(src/href)部分，我會需要從 HTML 中找出 href="..."、src="..."，我的程式參考了：

<https://en.cppreference.com/w/cpp/regex/ecmascript.html>

<https://en.cppreference.com/w/cpp/language/escape.html>

兩個連結的內容邏輯。

我的程式碼:std::regex rg("(src|href)\\s*=\\s*"([^\"]+)\\"", std::regex::icase);

部分	對應定應	意義
'(src	href)'	alternation + group
\s*	character class + quantifier	0或多個空白字元
=	字面符號	等號
\s*	同上	同上
"	字面符號	字面上的雙引號
([^\"]+)	Negated char class + quantifier	抓出引號內部的網址字串
"	同上	同上

4. 建立資料夾、檔案存在檢查

我參考了：

<https://en.cppreference.com/w/cpp/filesystem/exists.html>

<https://learn.microsoft.com/en-us/cpp/standard-library/filesystem-functions?view=msvc-170#exists>

兩個網站

(1) 檔案/資料夾是否存在

網頁程式區段	我的程式區段
<pre>bool exists(const std::filesystem::path& p);</pre>	<pre>inline void ensureDir(const std::string &path) { if (!std::filesystem::exists(path)) { std::filesystem::create_directories(path); } }</pre>

(2) 建立資料夾

微軟程式區段	我的程式區段
<pre>create_directory(const path& pval);</pre>	<pre>inline void ensureDir(const std::string &path) { if (!std::filesystem::exists(path)) { std::filesystem::create_directories(path); } }</pre>

5. 多執行緒 std::thread + std::mutex (同步保護)

觀念用到:

<https://en.cppreference.com/w/cpp/thread/thread.html>

<https://en.cppreference.com/w/cpp/thread/mutex.html>

兩個網站

保護對象	理由	模組檔案
tasks 任務 queue	多執行緒 push/pop 時避免 race condition	advanced.cpp
progress[]	多執行緒更新進度時避免資料錯亂	advanced.cpp
stats->mtx	更新統計數據避免競爭	downloader.cpp

6. HTML 網頁重寫(Rewrite)功能參考來源:

本專案的 HTML 重寫 (rewriteHTML) 功能並非抄襲他人程式，

其原理基於 HTML 與 URL 的官方標準規範，包含：

HTML <a> 的 href 屬性，來源為MDN Web Docs:

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/a>

HTML 的 src 屬性，來源為MDN Web Docs:

<https://developer.mozilla.org/en-US/docs/Web/HTML/Element/img>

URL 的相對 / 絕對路徑解析規則(用於 absoluteURL)，來源為MDN Web Docs:

https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL

URL Rewriting 概念 (將連結改寫)，來源為MDN Web Docs:

https://developer.mozilla.org/en-US/docs/Learn/Common_questions/Web_mechanics/What_is_a_URL#rewriting_urls

7. GetTickCount64() — Windows 計時 API

參考連結:<https://chatgpt.com/g/g-p-69296ac033188191abc75fb3d39b4e78-pa-chong-c/c/692aef9-d290-8324-8779-0bf90c46e01f>

參考了連結中的:ULONGLONG GetTickCount64();

8. 編譯 web.exe 指令參考:<https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html#index-o>

9. 閱讀線上書籍(裡面有更多教學):

Beej's Guide to Network Programming <https://beej.us/guide/bgnet/>

[補充: UML圖示是用[PlantUML Web Server](#)]

五、本次期末專題心得

這次的專題作業是我這學期中最具挑戰性的一份程式作業。剛開始看到題目時，其實有點被嚇到，因為要求不能使用任何現成的 HTTP 函式庫，而且是用 C++ 寫(不能用 python寫)，所有的 HTTP request、response、URL 解析、絕對與相對路徑轉換、HTML 下載與重寫、遞迴爬取、外部網站判斷、多執行緒下載、續傳機制、檔案大小與副檔名篩選等功能都要自己試著做出來。

在實作過程中，我遇到非常多錯誤，包括 Winsock 無法成功連線、recv() 回傳 -1、Content-Length 抓不到、遞迴深度錯亂、外站 HTML 被覆蓋、檔案重複下載、progress 同步問題等，一次次遇到這些 bug，我幾乎都要停下來重新檢查所有 code 流程，因為我有將功能拆成多個模組 code 再網綁一起做編譯，有可能一處改完，另外一處的 error 忘記修正，所以花了一訂的時間在做 debug 上(要從 URL 解析、到 HTML link 提取、再到任務 queue 與 worker 行為，一層一層找問題)。

總結來說，此次作業我對於用 C++ 做網路爬蟲這件事，有了一定的收穫，學習盪很多新知識點(在做作業、查找資訊的過程中學習到的)，